基于 Spark 的 AIOps 时间序列异常检测实验报告

2026 年 6 月

目录

1. 实验概述 1
2. 实验环境与数据 1
   2.1 平台环境 1
   2.2 数据与清洗 2
3. 四个异常检测方案 2
   3.1 V1：分组 IQR Baseline 2
   3.2 V2：分组 IQR + Z-score 3
   3.3 V3：Window Z-score 3
   3.4 V4：MultiMetric Fusion 4
4. 四个方案的递进关系 5
5. Spark 运行效率问题与优化 5
   5.1 一开始为什么很慢 5
   5.2 为什么“分成两段”后更快 5
6. 关于不同小组流程相似的说明 5
7. 小组成员及分工 6
8. 局限与结论 6

1. 实验概述

本项目完成课程第四题“时间序列异常检测”。目标是在无可靠异常标签的情况下，从大规模 AIOps 监控数据中识别异常点、局部异常和多指标共同异常。实验实现四个逐步扩展的方案：

```
V1 分组 IQR Baseline
  → V2 IQR + Z-score 严格筛选
  → V3 Window Z-score 局部时间检测
  → V4 MultiMetric Fusion 多指标融合
```

2. 实验环境与数据

2.1 平台环境

项目	配置
集群	1 个 Master，2 个 Worker
服务器	CentOS 7，每台 8 核 CPU、16GB 内存
Hadoop	3.3.6
Spark	3.5.1，Scala 2.12
Java	OpenJDK 8
HDFS	2 个 DataNode，副本数 2

原始数据约 3.5GB，上传至 /aiops/metric。HDFS 因副本数为 2，实际占用约 7.0GB。清洗阶段使用 Spark Scala 读写 HDFS；四个检测方案使用 Spark Standalone 集群运行。

2.2 数据与清洗

数据包含 2 天、3 个 cloudbed，共 6 个场景，指标分为 service、node、container、jvm、istio 五类。service 原始数据是一行包含 rr/sr/mrt/count 的宽表，其他类型基本为 timestamp/cmdb_id/kpi_name/value 长表。

Spark 清洗流程为：递归读取 CSV，识别指标类型，将时间戳转为 long、指标值转为 double，过滤关键字段为空或非法的记录，将 service 宽表转为长表，最后以 Parquet 写入：

```
hdfs://master:9000/aiops/output/clean_metrics
```

统一字段为：

```
timestamp, object_id, kpi_name, metric_type, value, source_file
```

object_id + kpi_name + metric_type 确定一条算法处理的指标序列。清洗结果如下：

metric_type	记录数
container	22,211,676
istio	13,695,910
jvm	5,750,910
node	2,933,643
service	380,008
总计	**44,972,147**

共形成 8,730 条时间序列。原始数据每 60 秒采样一次，一个完整的单日序列典型长度为 24×60=1,440 点。但清洗后按对象和 KPI 跨文件归组，且部分指标稀疏，因此实际序列长度并不统一；按总记录数与序列数计算，平均约为 5,151 点。

3. 四个异常检测方案

3.1 V1：分组 IQR Baseline

实现 V1 作为无监督基线。程序先筛选一个 metric_type，再按 object_id + kpi_name + metric_type 分组，使每条序列拥有独立阈值，避免将请求数、响应时间、内存等不同量纲混合计算。

```
Q1 = 25% 分位数
Q3 = 75% 分位数
IQR = Q3 − Q1
lower = Q1 − 1.5×IQR
upper = Q3 + 1.5×IQR
```

value 超出上下界且 IQR != 0 时判为异常。分位数使用 percentile_approx(..., 300) 以降低计算代价。分组统计表 iqr_stats 再按分组键 join 回原始点，输出 anomaly_points。

结果与分析

类型	异常点	异常序列
container	238,853	1,312
istio	167,447	658
jvm	85,374	97

类型	异常点	异常序列
node	73,792	148
service	9,407	15
总计	**574,873**	**2,230**

总异常率约 1.28%。V1 不需要标签，规则简单、可解释，但只依赖分位数边界，检出结果较宽，其中可能包含正常边界波动。

3.2 V2：分组 IQR + Z-score

实现　V2 沿用 V1 的分组粒度，在 Q1、Q3 之外计算每条序列的均值和标准差：

```
z = (value − mean) / std
```

当 std 为 0 或空时，该序列没有可用的标准差，不进行 Z-score 异常判断。最终采用严格规则：

```
IQR 异常 AND |z| > 3.0
```

3.0 参考常用的 3σ 经验阈值。由于数据没有真实标签，这是一个可解释的经验选择，不能解读为经过监督指标证明的最优值。程序对 stats 和打分结果使用 MEMORY_AND_DISK 持久化，以避免多个输出重复计算上游 DAG。

结果与分析

类型	V1 异常点	V2 异常点	V2/V1
container	238,853	40,554	16.98%
istio	167,447	64,184	38.33%
jvm	85,374	12,710	14.89%
node	73,792	11,144	15.10%
service	9,407	636	6.76%
总计	**574,873**	**129,228**	**22.48%**

V2 异常率约 0.287%。异常点减少 77.52%，说明 AND 规则有效收紧了 V1 的结果；它的目的是提高统计上的判定严格度，而不是证明减少的点一定是误报。istio 保留比例最高，说明其 IQR 异常中有更多点同时远离均值；service 只保留 6.76%，且 636 个点中主要为 mrt 响应时间异常。

V2 仍有两个明确不足：均值和标准差由整条序列计算，忽略局部时间变化；每次仍只判断一个 KPI，不能表示多指标联动。这两点分别引出 V3 和 V4。

3.3 V3：Window Z-score

实现　V3 对每条 object_id + kpi_name + metric_type 序列按 timestamp 排序，每个点只使用当前点和前 100 个历史点：

```
.rowsBetween(-100, 0)
```

在窗口中计算 window_mean、window_std 和 window_z_score，满足以下条件时判为局部异常：

```
window_point_count >= 30 AND |window_z_score| > 3.0
```

设置最少 30 点是为了避免序列起始阶段样本太少，均值和标准差不稳定。

为什么选择 100 个历史点　数据的采样间隔为 60 秒，因此前 100 点约对应最近 100 分钟，加上当前点后窗口最多 101 点。典型单日序列为 1,440 点，100 个历史点约占 6.9%，能保留“局部”含义；同时它明显大于最少 30 点，可以计算相对稳定的均值和标准差。

该值是结合采样频率、序列长度和计算代价作出的工程折中：过小的窗口易被瞬时噪声带动，过大的窗口又会向全局统计退化并增加窗口排序与计算开销。由于缺少标注，本实验不声称 100 是经过准确率验证的最优窗口，而将其作为可解释、可复现的默认值。

结果与分析

类型	V2 异常点	V3 异常点	变化
container	40,554	108,723	+168%
istio	64,184	84,068	+31%
jvm	12,710	6,876	-46%
node	11,144	21,260	+91%
service	636	3,585	+464%
总计	**129,228**	**224,512**	**+73.73%**

V3 异常率约 0.499%。`service`、`container` 和 `node` 明显增加，说明它们存在不容易被整段统计发现的局部波动；`jvm` 则减少，说明一些全局极端值在它们所在的局部阶段中并不突出。`service` 的 Top 序列中出现了多个 `count` 指标，表明局部窗口对短时请求量升降更敏感。V3 与 V2 是“局部敏感”和“全局严格”的互补，异常数增加不能单独证明准确率更高。

3.4 V4：MultiMetric Fusion

实现　V4 先为每个 `metric_type` 选择出现频率最高的 Top 30 KPI，再按 `timestamp + object_id + metric_type` 分组，用 `pivot(kpi_name)` 将多个 KPI 放到同一行。对每个 KPI 列计算全局 Z-score，统计同一行中满足 `|z| > 3.0` 的 KPI 数量：

```
anomaly_metric_count >= 2
```

满足条件时输出一个多指标融合异常事件，同时保存 `max_abs_z_score` 表示该事件中偏离最强的单个 KPI。当前实现使用统计融合而非 Isolation Forest，原因是 Spark MLlib 默认不包含 Isolation Forest，当前方案不需要额外第三方包，可直接复现。

结果与分析

类型	融合异常事件	涉及对象
container	27,172	81
istio	44,784	99
jvm	153	2
node	1,743	6
service	106	5
总计	**73,958**	-

V4 结果主要集中在 `istio` 和 `container`，说明容器资源与服务网格层更容易出现多个 KPI 同时偏离的状态。`service` 仅有 106 个融合事件，其中可见 `sr`、`rr`、`count`、`mrt` 同时异常的样例。

需特别说明：V1-V3 统计的是“单 KPI 异常点”，V4 统计的是“对象-时间粒度的多 KPI 异常事件”。因此 73,958 可用于展示结果规模和层级分布，不能与前三版异常点数直接解读为同口径的准确率升降。

4. 四个方案的递进关系

版本	异常单位	解决的问题	仍有局限
V1 IQR	单 KPI 时间点	建立无标签、分组、可解释基线	单一分位数规则偏宽
V2 IQR+Z-score	单 KPI 时间点	用两种统计规则交集收紧结果	仍是整条序列的静态统计
V3 Window Z-score	单 KPI 时间点	捕捉相对近期基线的局部突刺	仍未建模 KPI 之间的联动
V4 MultiMetric Fusion	对象-时间事件	识别同一对象多 KPI 共同异常	仍是统计融合，且缺失值填 0 可能影响结果

因此，这四版不是完全相同的算法排名，而是从“建立基线”到“提高严格度”，再向“时间局部性”和“指标关联性”扩展的能力路线。V3 和 V4 从不同方向弥补 V2，两者并非严格先后依赖。

5. Spark 运行效率问题与优化

5.1 一开始为什么很慢

初版脚本为了查看进度，在主检测流程中连续调用 `count()`、`show()`、`orderBy()` 和 Top N 统计。Spark DataFrame 采用惰性求值，上述每个 Action 都会触发一次作业；如果上游结果没有合理持久化，Spark 会再次执行读取、分组、分位数、join 等上游 DAG。

同时，数据总量接近 4500 万条，其中 `container` 和 `istio` 分别超过 2200 万和 1300 万条。`groupBy`、`join`、Window 排序和 `pivot` 都容易触发 shuffle，需要在节点间传输数据，并可能因内存不足发生磁盘 spill。运行中还曾出现 SparkSubmit 残留占用 Master 资源、Worker 失联等问题。因此慢的核心不是 IQR 公式本身，而是重复 Action、大规模 shuffle、资源不足和进程未释放的组合。

5.2 为什么“分成两段”后更快

后期将流程明确拆成两段：

阶段 `A`：只完成检测，将 `stats` 和 `anomaly_points` 物化写入 `HDFS`
阶段 `B`：检测结束后，另起轻量作业读取 `anomaly_points`，再做 `count/Top N/汇总`

阶段 A 不再为了打印中间结果触发多个 Action；写入 HDFS 后，复杂计算链已经物化。阶段 B 只读取过滤后的小异常集，不会因每个统计动作重算全量分位数、窗口或 join。因此“拆分”并没有减少必要的算法工作，而是避免了不必要的重复执行。

另外，主检测也按 `metric_type` 分批，每次只处理一类数据。这使单个作业的 shuffle、缓存和失败重试范围更小。`service/node`、`jvm/istio`、`container` 分别使用 16、32、64 个 shuffle 分区，V2/V3/V4 对重用中间结果使用 `MEMORY_AND_DISK`，在内存不足时允许落盘，避免直接 OOM。

6. 关于不同小组流程相似的说明

第四题给定了相同的原始数据、大数据平台和时间序列目标，因此各组容易收敛到类似的技术主干：HDFS 存储、Spark 清洗、长表统一、分组统计、滑动窗口和结果可视化。这些都是处理该类数据的常规选择，流程相似本身不意味实现和实验结果相同。

本组在方法检索、方案头脑风暴、Scala 脚本草拟和文档整理中使用了大模型作为辅助工具。实际实验中，我们仍根据数据结构、集群环境和运行结果进行了验证与修正：例如将“分组检测”前置到 V1，将 V2 确定为 AND 严格策略，根据 60 秒采样设置 V3 的 100 点窗口，并在 Spark MLlib 无内置 Isolation Forest 的条件下将 V4 落地为可复现的多指标 Z-score 融合。运行慢、内存压力、分批执行和结果口径也是实际运行后才根据现象调整的。

7. 小组成员及分工

组别：第六组

- **马蕊：时间动态异常检测与实验对比**。负责提出 V3 滑动窗口 Z-score 思路，解决 V1/V2 使用全局静态统计量、忽略时间局部变化的问题；完成 V3 运行验证，整理 V2/V3 在异常数量、异常序列和 service Top 异常上的差异；整理各版本 `anomaly_points`、`stats`、`summary` 等输出数据。
- **张玳瑗：实验结果统计与可视化数据构建**。负责设计 V1-V4 结果统计口径，整理输出数据；提取总异常数、按 `metric_type` 分布、Top 异常序列、V1/V2/V3/V4 对比表和时间序列展示数据；帮助构建前端 Dashboard 所需的 CSV/JSON 数据，并协助完成同一时间序列四版本异常标记对比。
- **李姝漫：多指标融合与结果展示分析**。负责提出 V4 从单指标检测扩展到多指标融合检测的思路，设计 `timestamp + object_id + metric_type` 粒度下的多 KPI 共同异常判断；完成 V4 结果分析，并将 V1-V4 结果转化为图表和前端 Dashboard 展示逻辑。
- **王咏心：原始数据分析与清洗方案设计**。负责分析老师提供的 6 个场景、5 类指标数据结构，提出将不同格式数据统一为 `timestamp / object_id / kpi_name / metric_type / value` 的长表结构；完成 service 宽表转长表、类型转换、空值过滤和 `clean_metrics` 构造方案，验证 V1-V4 数据来源正确。
- **吴柯蝶：项目统筹与核心算法实现**。提出“四阶段迭代式异常检测路线”：V1 建立 IQR Baseline，V2 加入 Z-score 提高置信度，V3 引入滑动窗口处理时间动态，V4 做多指标融合；负责 V1/V2 核心实现、结果整合和最终答辩主线设计。负责 Hadoop/HDFS/Spark 环境配置，完成原始数据上传、HDFS 路径验证、Spark 读取测试，并协助解决运行慢、内存压力和参数配置问题。

8. 局限与结论

受数据标注和实验条件限制，本实验主要从检测结果、方案特点和实际运行情况对四个版本进行分析，相关参数和数据处理策略仍有进一步验证与优化的空间。

总体上，项目完成了从 HDFS 存储、Spark 统一清洗到四类异常检测能力的全流程。V1 提供可解释基线，V2 得到更严格的交集结果，V3 补充局部时间敏感性，V4 将判断单位扩展到多 KPI 共同事件。四者共同形成了从单点统计到时间与多指标关系的递进式方案。